

Methodology

John B. Cole

2025-12-16

1 Preface

1.1 Acknowledgments

PyPedal was initially written to support the author's dissertation research while at [Louisiana State University](#). The initial development was supported in part by a grant from The Seeing Eye, Inc. (Morristown, NJ). The author wishes to thank Paul VanRaden for many helpful suggestions for improving the ability of PyPedal to handle computations in large pedigrees. Additional feedback in the form of bug reports, feature requests, and discussion of computing strategies was provided by Dan Cieslak, Bradley J. Heins (University of Minnesota), Edward H. Hagen (Institute for Theoretical Biology, Humboldt-Universität zu Berlin), Kathy Hanford (University of Nebraska), Thomas Kelly (Department of Animal and Poultry Science, University of Guelph), Thomas von Hassell, Gianluca Saba, and Ken Severin (University of Alaska – Fairbanks). Gregor Gorjanc wrote a blog entry describing [how to install PyPedal on Debian Linux](#). Fernando Perez posted a LaTeX preamble to the NumPy listserver that dramatically improved the PDF version of this manual.

The Newfoundland pedigree was taken from the [Newfoundland Dog database](#) and is used with permission.

The pedigree of European royalty used in the GEDCOM discussion, [ged3.ged](#), was taken from the [Genealogy Forum website](#). It is believed to be in the public domain, and is used with the knowledge of the website administrators.

2 NAME: introduction.qmd

Any sufficiently disguised bug is indistinguishable from a feature. — Rich Kulawiec

2.1 What is PyPedal?

This chapter introduces PyPedal, a Python-language tool for working with pedigrees.

PyPedal (**P**ython **P**edigree **A**nalysis) is a tool for analyzing pedigree files. It calculates several quantitative measures of genetic diversity from pedigrees, including average coefficients of inbreeding and relationship, effective founder numbers, and effective ancestor numbers. Checks are performed catch common mistakes in pedigree files, such as parents with more recent birthdates or smaller ID numbers than their offspring and animals appearing as both sires and dams in the pedigree. Tools for pedigree visualization and report generation are also provided. PyPedal only makes use of information on pedigree structure, not individual genotypes. Genotypes can be assigned to founders for use in gene-dropping simulations to calculate the effective number of founder genomes.

PyPedal is a [Python](#) language module that may be called by programs or used interactively. You must have Python 3.11 (or later) installed in order to use PyPedal. The NumPy module must also be installed in order for you to use PyPedal. In addition, there are several third-party packages used by PyPedal, which are discussed in the chapter on installing PyPedal.

This manual is the official documentation for PyPedal. It includes a tutorial and is the most authoritative source of information about PyPedal with the exception of the source code. The tutorial material will walk you through a series of manipulations of a simple pedigree. All users of PyPedal are encouraged to follow the tutorial with a working installation.

2.2 Features

Key features of PyPedal include:

- Reading pedigree files in user-defined formats;
- Checking pedigree integrity (duplicate IDs, parents younger than offspring, etc.);
- Generating summary information such as frequency of appearance in the pedigree file;

- Reordering and renumbering of pedigree files;
- Computation of the numerator relationship matrix (A) from a pedigree file using the tabular method;
- Inbreeding and relationship calculations for large pedigrees (pedigrees up to 600,000 animals have been processed with PyPedal);
- Computation of coefficients of inbreeding using the Meuwissen and Luo (1992) and modified Meuwissen and Luo (Quaas, 1995, unpublished) algorithms;
- Computation of average total and individual coefficients of inbreeding and relationship;
- Calculation of coefficients of partial inbreeding using an iterative tabular method (Lacy, Alaks, and Walsh 1996; Gulisija et al. 2006);
- Calculation of coefficients of ancestral inbreeding using the methods of Ballou (1997) or Suwanlee et al. (2007);
- Decomposition of A into T and D such that $A = TDT'$;
- Computation of the direct inverse of A (not accounting for inbreeding) using the method of Henderson (1976);
- Computation of the direct inverse of A (accounting for inbreeding) using the method of Quaas (1976);
- Storage of A and its inverse between user sessions as persistent Python objects using the `pickle` module to avoid unnecessary calculations;
- Calculation of theoretical and actual effective population sizes;
- Computation of effective founder number using the exact algorithm of Lacy (1989);
- Computation of effective founder number using the approximate algorithm of Boichard et al. (1997);
- Computation of effective ancestor number using the algorithms of Boichard et al. (1997);
- Selection of subpedigrees containing all ancestors of an animal;
- Identification of the common relatives of two animals;
- Calculation of the inbreeding of offspring from a prospective mating;
- Output to ASCII text files, including matrices, coefficients of inbreeding and relationship, and summary information;
- Importation and exportation of GEDCOM 5.5 files;
- Importation and exportation of GENES 1.20 (dBase III) files;
- Loading pedigrees from, and saving them to, MySQL, Postgres, and SQLite databases;
- Simulation of pedigrees using an algorithm derived from that used in Matvec 1.1a;
- Set operations on pedigrees (unions and intersections);

PyPedal has been used to perform calculations on pedigrees as large as 600,000 animals and has been used in scientific research (Cole, Franke, and Leighton 2004; Cole 2007).

2.3 Where to get information and code

Code and documentation are available from the [PyPedal 3 Github repository](#).

3 NAME: getting_started.qmd

4 VERSION: 3.0.0 (16DECEMBER2025)

**5 AUTHOR: John B. Cole
(john.b.cole@gmail.com)**

6 LICENSE: LGPL v2.1 (see LICENSE file)

For an example of how to load a pedigree, see Listing 6.1.

Listing 6.1

```
from PyPedal import pyp_newclasses
options = {
    'renumber': 0,
    'pedfile': '../test_files/mrode.ped',
    'pedformat': 'asd',
    'pedname': 'Mrode Pedigree Table 2.1',
    'pedigree_is_renumbered': 1,
    'messages': 'quiet',
}
result = pyp_newclasses.load_pedigree(options)
result.metadata.printme()
```

Metadata for Mrode Pedigree Table 2.1 (../test_files/mrode.ped)

Records:	6
Unique Sires:	3
Unique Dams:	2
Unique Gens:	1
Unique Years:	1
Unique Founders:	2
Unique Herds:	1
Pedigree Code:	asd

7 NAME: input-output.qmd

The only legitimate use of a computer is to play games. — Eugene Jarvis

Getting data into and out of programs, while extremely important to end-users, is often challenging. PyPedal can load pedigrees from — and save them to — many different sources. A list of supported input and output formats is shown in Table Table 7.1.

Table 7.1: PyPedal input and output methods.

Direction	Source	Description
Input	db	Load an ASDx-formatted data from an SQLite database
	from-graph	Load a pedigree from an instance of an XDiGraph object (not a file)
	gedcom-file	Load a pedigree from a GEDCOM 5.5 file
	graphfile	Load a pedigree from an adjacency list using the <code>read_adjlist()</code> function from NetworkX
	simulate	Simulate a pedigree rather than loading it from a file
Output	textstream	Load a pedigree from a string containing animal records
	save	Save a pedigree with user-specified filenames, pedigree format codes, and separators
	savedb	Save a pedigree to a database table
	savedged-com	Save a pedigree to a GEDCOM 5.5 file
	savedged-com	Save a pedigree to a GENES 1.20 file (DBASE III format)
	savegraph	Save a pedigree to a file as an adjacency list

It has evolved over time that the input methods are implemented as cases of `pedsource` in the `NewPedigree::load()` method, while output methods are implemented as individual methods of `NewPedigree`. While it is quite straightforward to add new methods for pedigree output, it is trickier to add new input sources. The greater difficulty in adding new input sources is largely due to the mysterious workings of the `NewPedigree::preprocess()` method, which

walks through input line-by-line to load the pedigree and perform a number of integrity checks. Once the data are loaded into a `NewPedigree` object it's easy to output them because there's no need to check the integrity of the pedigree and relationships among the records in the pedigree. If you want to implement a new input source the sanity-saving way to go is to get the data into a list that you can pass to `preprocess()`; `preprocess()` can then walk the list as it would walk through the lines of an input file.

Database operations are discussed in their own section at the end of this chapter.

7.1 Input

The process of data input has been made as simple as is reasonably possible, but it is ultimately the responsibility of the user to prepare their data. Most people will load their data from a simple text file, with one pedigree entry per line. A number of data integrity checks are performed when a pedigree is loaded. First, the pedigree format string is checked for validity and compared to a line of data to make sure that the specified number of columns of data exist. Once that check is passed, individual records are loaded and checked one at a time. As individual records are validated they're added to the pedigree. Duplicate records are eliminated, parents that appear in the pedigree but do not have their own record in the input data are added to the pedigree, and sexes are checked (animals cannot appear as both males and females). If animal IDs are strings they're hashed to get numeric IDs for use in reordering and renumbering.

If you used an earlier version of PyPedal you may have added a pedigree format string to your pedigree file. You don't need to include that string in the current version of PyPedal, and if `NewPedigree::preprocess()` sees one while reading a pedigree file it will ignore it. You may include comment lines in your file as well; they will also be ignored.

7.1.1 Graph Objects

Pedigrees can be represented as a type of mathematical object called a directed graph (digraph; not to be confused with visualizations of data). The `NetworkX` module provides Python tools for working with digraphs, and PyPedal provides a convenient way for loading a pedigree from an instance of an `XDiGraph` object.

```
example = pyp_newclasses.loadPedigree(optionsfile='new_networkx.ini')
ng = pyp_network.ped_to_graph(example)
options = {}
options['pedfile'] = 'dummy'
options['pedformat'] = 'asd'
example2 = pyp_newclasses.loadPedigree(options,pedsource='graph',pedgraph=ng)
```

```
example2.metadata.printme()
```

You can see from the printout of the metadata from the new pedigree, `example2`, that the graph `ng` was successfully converted to a `NewPedigree` object.

```
Metadata for Testing fromgraph() (dummy)
```

```
Records:                13
Unique Sires:            3
Unique Dams:             4
Unique Gens:             1
Unique Years:            1
Unique Founders:        5
Unique Herds:            1
Pedigree Code:          asd
\end{verbatim}
'''
```

GEDCOM Files

A thorough description of support for GEDCOM files may be found in the Appendix.

GENES Files

A thorough description of support for GENES files may be found in the Appendix.

Text Files

Most users will load their pedigrees from simple text files. As an example, consider a large pedigree which is presented below.

```
```{python}
#| eval: false # Don't evaluate this chunk
#| echo: true # Do show this chunk in the final rendered document
dogID,fatherID,motherID,gender,born
64 66 67 2 1979
63 64 65 1 1982
62 191 195 2 1982
61 64 65 2 1982
...
```
```

This pedigree can be loaded using this short program:

```

\begin{verbatim}
options = {
    'pedfile': 'dog.ped',
    'pedname': 'A Large Dog Pedigree',
    'pedformat': 'asdgb',
}
if __name__ == '__main__':
    test = pyp_newclasses.loadPedigree(options)

```

There are examples of loading pedigrees from text files throughout this manual.

7.1.2 Text Streams

There are some use cases, such as web services, for which it may be desirable to load pedigrees from strings rather than from files. This is done by passing the `pedsource` keyword to `pyp_newclasses.load_pedigree()` with a value of `textstream`, along with a string named `pedstream`:

```

options = {}
options['pedfile'] = ''
options['messages'] = 'verbose'
options['pedformat'] = 'ASD'

if __name__ == "__main__":
    pedstream = 'a1,s1,d1\na2,s2,d2\na3,a1,a2\n'
    test = pyp_newclasses.loadPedigree(options, pedsource='textstream', pedstream=pedstream)

```

Only ASD-formatted pedigrees can be loaded this way, individual IDs are separated with commas, and successive records are separated by newlines. These restrictions are hard-coded into the `NewPedigree::load()` method. All records must contain a newline, including the last record in the string! You must also set the `pedfile` option to some value, even if that value is just an empty string, as in the example above.

The expected use case is that the pedigree would be retrieved from a database, and the result set from the SQL query converted into a string. There's an upper bound on the size of pedigree that can be loaded from a stream based on the physical memory available on your platform, and extremely large pedigrees should be loaded from a text file.

7.2 Output

7.2.1 GEDCOM Files

A thorough description of GEDCOM file output may be found in the Appendix.

7.2.2 GENES Files

A thorough description of GENES file output may be found in the Appendix.

7.2.3 Graph Objects

`NewPedigree::savegraph()` saves a pedigree to a file as an adjacency list, which is a commonly-used format for describing digraphs. If the pedigree has already been converted to a digraph, pass it using the `pedgraph` argument; otherwise, the method will call the appropriate converter:

```
test.savegraph(pedoutfile='test.adj')
```

The file `test.adj` has the following contents:

```
# sqlite.py
# GMT Tue Mar  4 20:38:52 2008
# Text Stream
1 5
2 6
3 5
4 6
5 7
6 7
7
```

This example shows there can be a considerable loss of information when going from a PyPedal pedigree to some other way of representing the pedigree, such as a graph.

7.2.4 Text Files

The `NewPedigree::save()` method writes a PyPedal pedigree to a user-specified file based on a pedigree format string and optional column separator provided by the user. By default, the pedformat is `asd` and the sepchar is `.` Any attribute of a `NewAnimal` object can be written to an output file.

`NewPedigree::save()` tries to never overwrite your data. If you don't pass a filename argument a file whose name is derived from, but not the same as, the original pedigree filename will be used. The string `_saved` will be appended to the filename in order to distinguish it from the original pedigree file.

There are a few points that you need to be aware of when using `NewPedigree::save()`. First, you may use any column separator you choose **except** the empty string (`sepchar=''`); if you try and do that the `sepchar` will be changed to the global default (currently `,`), and a message written to the log. This means there's no way for you to run all the columns in your pedigree file together such that they cannot easily be separated. Sorry. You can use any other string you like to separate your columns (spaces, columns, and tabs have been tested). It **is** possible to write an incomplete pedigree file, that is, a file which does not include animal, sire, and dam information. PyPedal checks the `pedformat` to see if it contains either `asd` or `ASD` and warns you if it doesn't, but it'll still write the file you tell it to. The burden is on you, the user, to make sure your output file contains all the information you want it to. Finally, you can write whatever attribute you like to whichever column you like in the output file. For the sake of your sanity I strongly recommend that you always place the animal, sire, and dam IDs in the first three columns, but you're an adult and may do what you like.

Maybe a couple of examples will help clear that up. Here's the simplest case, taking defaults:

```
test.save('test_save_asd.ped')
```

gets you the result:

```
# test_save_asd.ped created by PyPedal at Tue Sep 28 16:39:36 2010
# Current pedigree metadata:
#     pedigree file: test_save_asd.ped
#     pedigree name: Untitled
#     pedigree format: asd
#     NOTE: Animal, sire, and dam IDs are RENUMBERED IDs, not original IDs!
# Original pedigree metadata:
#     pedigree file: ../simulated_pedigree.ped
#     pedigree name: Untitled
#     pedigree format: asdxg
1 0 0
2 0 0
3 0 0
4 0 0
5 0 0
...
21 19 10
22 9 12
```

```
23 1 10
24 1 21
25 13 8
```

Note the friendly header to tell you what you've got and where it came from. The note about renumbered IDs is particularly important – if the `pedigree_is_renumbered` flag is set then you will get this message to alert you that comparing this file against your original source file may give confusing results. They're not really conflicting, but if you compare renumbered with original IDs then you may assume there's some kind of problem when really there is not. Original and renumbered pedigrees are mathematically equivalent assuming there were no errors in the original file that were corrected by PyPedal. Isn't that reassuring? From here on out I will not show the header, and will include only the first five rows of data in the file for the sake of brevity (like I've ever let that stop me before). You can get the same thing as a CSV using the `sepchar` argument:

```
test.save('test_save_asd_csv.ped', sepchar=',')
```

which produces:

```
1,0,0
2,0,0
3,0,0
4,0,0
5,0,0
...
```

Sometimes you want to go all the way and dump everything into a file. This is like one of those pizzas with every variety of cured meat known to food science piled up on top. Here you go:

```
test.save('test_save_combo_all.ped', pedformat='asdgxbfrnylepASDLhHu')
```

which gives you what you asked for, even if many of the values aren't particularly interesting:

```
1 0 0 0 m 01011900 0.0 Unknown_Breed 7 1900 0 -999 -999.0 7 Unknown_Name Unknown_Name ['19
2 0 0 0 m 01011900 0.0 Unknown_Breed 6 1900 0 -999 -999.0 6 Unknown_Name Unknown_Name ['19
3 0 0 0 m 01011900 0.0 Unknown_Breed 5 1900 0 -999 -999.0 5 Unknown_Name Unknown_Name ['19
4 0 0 0 f 01011900 0.0 Unknown_Breed 3 1900 0 -999 -999.0 3 Unknown_Name Unknown_Name ['19
5 0 0 0 f 01011900 0.0 Unknown_Breed 2 1900 0 -999 -999.0 2 Unknown_Name Unknown_Name ['19
...
```

Okay, I get it. You're like the Nigel Tufnel of Missouri – you want to see something that goes to 11. I pondered this for a while. You've read this far and deserve to be rewarded. So I tried

to think of the most awesomest thing that I could think of. So here you go, LSU fans, straight from Death Valley by way of Beltsville, MD:

```
test.save('test_save_peterson.ped', pedformat='asd', sepchar='...Kneel before Zod!...')
```

Maybe Patrick Peterson is really into genealogy – you never know – stop looking at me like that! This is a perfectly cromulent example of the extremes to which the `sepchar` argument can be pushed:

```
1...Kneel before Zod!...0...Kneel before Zod!...0
2...Kneel before Zod!...0...Kneel before Zod!...0
3...Kneel before Zod!...0...Kneel before Zod!...0
4...Kneel before Zod!...0...Kneel before Zod!...0
5...Kneel before Zod!...0...Kneel before Zod!...0
...
```

You’re thinking, “My eyes! Why would he do that?!” The same reason Zod gave us a little Heisman pose in the end zone after returning a West Virginia kickoff for 6 – because I can. (This kind of writing is probably going to get me in trouble one day, but the boss is out of town and there’s no one here to rein-in my behavior!) Wait, though. It gets better. PyPedal can get all Ourboros on this unholy creation:

```
# But, wait, can this actually work?
options2 = options
options2['pedfile'] = 'test_save_peterson.ped'
options2['sepchar'] = '...Kneel before Zod!...'
options2['pedformat'] = 'asd'
test2 = pyp_newclasses.loadPedigree(options2)
```

When you run this bit of code you see that it poses no problem at all for PyPedal. Perhaps political dissidents can communicate secretly by doing strange things with `sepchar`, although it’s more likely that PhD students will find a clever way to abuse MS students with it, instead.

```
[INFO]: Logfile untitled_pedigree.log instantiated.
[INFO]: Preprocessing test_save_peterson.ped
[INFO]: Opening pedigree file test_save_peterson.ped
      [INFO]: Renumbering pedigree at Wed Sep 29 13:42:05 2010
            [INFO]: Reordering pedigree at Wed Sep 29 13:42:05 2010
            [INFO]: Renumbering at Wed Sep 29 13:42:05 2010
            [INFO]: Updating ID map at Wed Sep 29 13:42:05 2010
      [INFO]: Assigning offspring at Wed Sep 29 13:42:05 2010
[INFO]: Creating pedigree metadata object
```



```

[INFO]: Instantiating a new PedigreeMetadata() object...
[INFO]: Naming the Pedigree()...
[INFO]: Assigning a filename...
[INFO]: Attaching a pedigree...
[INFO]: Setting the pedcode...
[INFO]: Counting the number of animals in the pedigree...
[INFO]: Counting and finding unique sires...
[INFO]: Counting and finding unique dams...
[INFO]: Setting renumbered flag...
[INFO]: Counting and finding unique generations...
[INFO]: Counting and finding unique birthyears...
[INFO]: Counting and finding unique founders...
[INFO]: Counting and finding unique herds...
[INFO]: Detaching pedigree...
Metadata for Untitled (test_save_peterson.ped)
Records:                25
Unique Sires:           7
Unique Dams:            7
Unique Gens:            1
Unique Years:           1
Unique Founders:        6
Unique Herds:           1
Pedigree Code:          asd

```

How's that for the strangest thing you're likely to see all day? I'm not suggesting that you seriously use column separators the way I just did, but this example shows that you can do some unusual things with PyPedal, and perhaps you'll come up with something very useful because of that flexibility. If nothing else, you now have a greater appreciation of what you can do with a dynamic language and questionable processing of user inputs. Remember, never trust any data a user gives you. They're a shifty lot, users.

7.2.5 Text Streams

`NewPedigree::tostream()` returns a text stream from an instance of a `NewPedigree` object. The text stream contains an ASD-formatted pedigree as a string in which individual IDs are separated by commas and successive records are separated by newlines.

```

>>> pedstream = 'a1,s1,d1\na2,s2,d2\na3,a1,a2\n'
>>> test = pyp_newclasses.loadPedigree(options,pedsource='textstream',pedstream=pedstream)
>>> pedstream2 = test.tostream()
>>> print pedstream2

```

```
'd1,0,0\nd2,0,0\ns1,0,0\ns2,0,0\na2,s2,d2\na1,s1,d1\na3,a1,a2\n'
```

Note that in this example the input and output text streams differ because the input stream did not include pedigree entries for all animals in the pedigree (for example, sire **s1**). Recall that PyPedal adds missing entries for parents when the pedigree is loaded.

7.2.6 Databases

PyPedal is able to load pedigrees from, and save them to, three databases: MySQL, Postgres, and SQLite. PyPedal uses the pyDAL database abstraction library (<https://github.com/web2py/pydal>) to talk to these databases. Additional databases are supported by pyDAL and require only minor changes to the connection strings in the `connectToDatabase` function in the `pyp_db` module in order to work with PyPedal as well.

A list of the database-related keywords may be found in Table Table 7.2.

Table 7.2: Database access parameters and their default values.

| Parameter | Default | Description |
|-------------------|-------------|---|
| database_debug | False | Toggle debugging messages in the database module on and off. |
| database_host | 'localhost' | The server on which the database is running. |
| database_name | 'pypedal' | The name of the database to be used. |
| database_password | 'anonymous' | The password needed to connect to your database. |
| database_port | '' | The port on which your database is listening for connections. |
| database_table | 'filetag' | The name of the database table to which the current pedigree will be written. |
| database_type | 'sqlite' | The type of database you are using ('mysql') |
| database_user | 'anonymous' | The name of a user with access rights to your database. |

7.2.7 Input from Databases

PyPedal can load ASDx-formatted pedigrees from an SQLite database using `pedsource='db'`. The pedigree will be loaded from the database and table specified in the `database_name` and `dbtable_name` variables. Consider the following example:

```
test = pyp_newclasses.loadPedigree(options,pedsource='db')
test.metadata.printme()
```

This produces the output:

```
Metadata for DB Stream ()
Records:          7
Unique Sires:     3
Unique Dams:      3
Unique Gens:      1
Unique Years:     1
Unique Founders:  4
Unique Herds:     1
Pedigree Code:    ASDx
```

Note that user-supplied values of the pedigree format string will be over-written by the `load()` method and don't affect database processing. Database importation is hard-coded to accept only pedigrees in that format.

It's possible to load pedigrees from a database with a format different than that expected by PyPedal, but it requires that you make changes to the `NewPedigree` class's `load()` method. Consider the database-loading code that's located at about line 826 of `pyp_newclasses.py`:

```
elif pedsource == 'db':
    self.kw['pedformat'] = 'ASDx'
    self.kw['sepchar'] = ','
    ...
    try:
        # Connect to the database
        if pyp_db.does_table_exist(self):
            conn = pyp_db.connect_to_database(self)
            if conn:
                sql = 'SELECT * FROM %s' % ( self.kw['database_table'] )
                dbstream = conn.GetAll(sql)
```

In order to load data from your own database you must change the pedigree format string to match the pedigree you want to load (see Table `tbl-pedigree-format-codes` for a list of pedigree format codes). You then need to change the SQL statement to match the column names in your database and the order of fields in your pedigree format string. The animal, sire, and dam IDs, respectively, should always be the first three columns in your records. The query results are stored as a list of tuples in `dbstream` and will be unpacked in the `preprocess()` method.

Please note that I'll try and answer questions about the way in which PyPedal handles loading pedigrees from databases, I cannot answer detailed questions about databases to which I don't have access. You should carefully test your custom queries in SQL to ensure they're working

correctly before adding them to PyPedal. Also, please check your pedigree format string to make sure it matches your query.

7.2.8 Output to Databases

PyPedal can write `NewPedigree` pedigrees to ASDx-formatted SQLite tables using the `NewPedigree::savedb()` method. The following program will result in the creation of a table named `test_save` in the database `test_pypedal_save`.

```
test.kw['database_name'] = 'test_pypedal_save'
test.kw['dbtable_name'] = 'test_save'
test.savedb()
```

The pedigree is saved to the database and table specified in the `database_name` and `dbtable_name` variables. If the specified table already exists then the records from the current pedigree will be added to the table. If you want to overwrite the existing database then you need to pass `drop=True` in the call to `savedb()`:

```
test.savedb(drop=True)
```

While this may strike some users as unexpected behavior it is based on the idea that you should never lose data unexpectedly. If you use only the defaults you'll never lose data. However, if you try and save a renumbered pedigree on top of another renumbered pedigree you'll run into problems because animal IDs are used as primary keys, and you'll have constraint violations.

It's possible to save pedigrees to a database with a format different than that expected by PyPedal, but it's a little harder than changing pedigree input. Changing the output table requires that you change the `NewPedigree` class's `savedb` method. Consider the code that begins on or about line 1412 of `pyp_newclasses.py` in `savedb`:

```
sql = 'create table %s ( \
    animalName    varchar(128) primary key, \
    sireName      varchar(128), \
    damName       varchar(128), \
    sex           char(1) \
);' % ( self.kw['database_table'] )
```

You need to rewrite this statement to add or remove columns; the SQL for a table that includes all fields in a `NewAnimal` object may be found in the `create_pedigree_table` function of the `pyp_db` module. Note that not every field in a `NewAnimal` object has a corresponding pedigree format code. Consult Table `tbl-pedigree-format-codes` for appropriate codes.

Once you've updated the SQL used to create your custom pedigree table you'll also need to change the code which prepares the input for writing to the database. This code is found on or about line 1443 in `savedb`:

```
sql = "INSERT INTO %s ( animalName, sireName, damName, sex ) VALUES ('%s', '%s', '%s', '%s'"
```

Note that you'll need to make appropriate casts yourself. For example, you'll want to cast things that are stored as integers or reals to make sure that you don't violate a column constraint. You must enclose strings in single-quotation marks (`' '`) yourself.

By default PyPedal saves pedigrees in `ASD` format. If you want to save your pedigree in `asd` format then you'll need to change the code on or about line 1436 from:

```
an = p.name
si = p.sireName
da = p.damName
```

to:

```
an = p.animalID
si = p.sireID
da = p.damID
```

It's better to add a new method supporting your output format to `NewAnimal` rather than changing the `savedb` method.

8 NAME: calculations.qmd

Are you quite sure that all those bells and whistles, all those wonderful facilities of your so called powerful programming languages, belong to the solution set rather than the problem set? — Edsger Dijkstra

8.1 Overview

In `?@sec-inputoutput` you learned how to get your pedigree data loaded into PyPedal. This chapter will show you what you can do with the pedigree once it's loaded. The examples in this chapter assume that you're working with a reordered and renumbered pedigree (see Section 10.1 for additional details).

In order to get the most out of your pedigrees you need to understand the basic structure of a PyPedal pedigree, which consists of two components: the pedigree itself, which is composed of a list of `NewAnimal` objects, and metadata, which is information about the pedigree itself. Some calculations are performed on the animal records directly, while others use the metadata, or some combination of the two. The fundamental goal of PyPedal is to provide the user with tools for asking questions about their pedigrees.

The following discussion will use the following pedigree taken from Boichard et al. (1997):

```
# pedformat: asdg
1 0 0 1
2 0 0 1
3 0 0 1
4 0 0 1
5 1 2 2
6 3 4 2
7 5 6 3
8 5 6 3
9 5 6 3
10 5 6 3
11 5 6 3
12 5 6 3
```

```
13 5 6 3
14 5 6 3
```

Many of the subsequent code snippets are taken from the `new_methods.py` example program.

8.2 Inbreeding and Relationships

```
inbr = pyp_nrm.inbreeding(example)
print('inbr: ')
>>> inbr: {
  'fx': {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0,
        8: 0.0, 9: 0.0, 10: 0.0, 11: 0.0, 12: 0.0, 13: 0.0, 14: 0.0},
  'metadata': {
    'nonzero': {'f_max': 0.0, 'f_avg': 0.0, 'f_rng': 0.0,
               'f_sum': 0.0, 'f_min': 0.0, 'f_count': 0},
    'all': {'f_max': 0.0, 'f_avg': 0.0, 'f_rng': 0.0, 'f_sum': 0.0,
            'f_min': 0.0, 'f_count': 14}
  }
}
```

The dictionary returned by `inbreeding()` contains two dictionaries: `fx` contains coefficients of inbreeding keyed to animal IDs, and `metadata` contains summary information about the coefficients of inbreeding in the pedigree. `metadata` also contains two dictionaries: `nonzero` contains summary statistics only for animals with non-zero coefficients of inbreeding, while `all` contains statistics for all animals.

Relationship metadata, similar to the inbreeding metadata described above but for coefficients of relationship, are available but not calculated by default.

```
inbr, reln = pyp_nrm.inbreeding(example,rels=1)
print('reln: ', reln)
>>> reln: {'r_nonzero_count': 10, 'r_nonzero_avg': 0.40000000000000002,
           'r_min': 0.25, 'r_sum': 4.0, 'r_avg': 0.19047619047619047, 'r_max': 0.5,
           'r_count': 21, 'r_rng': 0.25}
```

The dictionary of relationship metadata returned by `inbreeding()` also contains statistics for zero and non-zero coefficients of relationship. In the example presented above the pedigree contains...

Relationship metadata are not guaranteed to be correct when `method = 'vanraden'` is used. That's because `inbreeding_vanraden()` uses a speed-up when there are full-sibs in the pedigree to avoid repeating calculations. The metadata should be reasonably accurate for pedigrees with

few or no full-sibs. The summary statistics will not be very accurate in the case of pedigrees that contain many full-sibs.

The relationship metadata don't include individual pairwise relationships. In order to associate those with your pedigree you must create a `NewAMatrix` object, form the numerator relationship matrix (NRM), and attach it to the pedigree:

```
options = {}
...
example = pyp_newclasses.loadPedigree(options)
example.nrm = pyp_newclasses.NewAMatrix(example.kw)
example.nrm.form_a_matrix(example.pedigree)
```

If you know when you load a pedigree file that you want to calculate and store the NRM you can save a little typing by setting the `form_nrm` option:

```
options = {}
options['form_nrm'] = 1
...
example = pyp_newclasses.loadPedigree(options)
```

If you want to inspect the NRM you can use `example.nrm.printme()` to print the matrix to the screen, which is probably not a particularly good idea for large pedigrees.

```
[[ 1.  0.  0.  0.  0.5 0.  0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25]
 [ 0.  1.  0.  0.  0.5 0.  0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25]
 [ 0.  0.  1.  0.  0.  0.5 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25]
 [ 0.  0.  0.  1.  0.  0.5 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25]
 [ 0.5 0.5 0.  0.  1.  0.  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ]
 [ 0.  0.  0.5 0.5 0.  1.  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 1.  0.5 0.5 0.5 0.5 0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 1.  0.5 0.5 0.5 0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 1.  0.5 0.5 0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 0.5 1.  0.5 0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 0.5 0.5 1.  0.5 0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 0.5 0.5 0.5 1.  0.5 0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 1.  0.5 ]
 [ 0.25 0.25 0.25 0.25 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 1. ]]
```

If you want to get the pairwise relationship between two animals you need to use their renumbered IDs and subtract 1 (because the array is zero-indexed). For example, if you wanted the coefficient of relationship between animals 2 and 5 (an individual and its sire) you would use the indices 1 and 4:


```
print(example.nrm.nrm[1][4])
>>> 0.5
```

The NRM is symmetric, which means that `nrm[1][4]` and `nrm[4][1]` are identical.

```
print(example.nrm.nrm[1][4])
>>> 0.5
print(example.nrm.nrm[5][1])
>>> 0.5
```

You can also easily save the NRM to a file for future reference:

```
example.nrm.save('Amatrix.txt')
```

You can verify that the contents of `Amatrix.txt` are:

```
1.0  0.0  0.0  0.0  0.5 0.0 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.0  1.0  0.0  0.0  0.5 0.0 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.0  0.0  1.0  0.0  0.0 0.5 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.0  0.0  0.0  1.0  0.0 0.5 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.5  0.5  0.0  0.0  1.0 0.0 0.5  0.5  0.5  0.5  0.5  0.5  0.5  0.5
0.0  0.0  0.5  0.5  0.0 1.0 0.5  0.5  0.5  0.5  0.5  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 1.0  0.5  0.5  0.5  0.5  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  1.0  0.5  0.5  0.5  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  1.0  0.5  0.5  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  0.5  1.0  0.5  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  0.5  0.5  1.0  0.5  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  0.5  0.5  0.5  1.0  0.5  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  0.5  0.5  0.5  0.5  1.0  0.5
0.25 0.25 0.25 0.25 0.5 0.5 0.5  0.5  0.5  0.5  0.5  0.5  0.5  1.0
```

Information on the endianness and precision of the data in the array are lost, so this is not a good way to archive data or move data between machines with different endianness. The data are always written in C (row-major) order. For better performance you can set the `nrm_format` option to `binary`, which will use a binary file format instead of text.

Finally, if you're going to work on the exact same pedigree later you can load `Amatrix.txt` and avoid having to recalculate the NRM entirely:

```
example.nrm2 = pyp_newclasses.NewAMatrix(example.kw)
example.nrm2.load('Amatrix.txt')
example.nrm2.printme()
```

Since we've loaded `Amatrix.txt` into a second NRM (`nrm2`) attached to our pedigree it's straightforward, if tedious, to verify that the two NRM contain the same values.

8.3 Matings

There are many questions you might wish to ask that involve matings, such as, “What’s the minimum-inbreeding mating among this set of individuals?”. Several routines in `pyp_metrics`...

Suppose you're considering a mating between animals 5 (index 4) and 14 (index 13), which is a sire-daughter mating. How would you go about this? You simply call `pyp_metrics.mating_coi()`:

```
print '\tCalling mating_coi() at %s' % ( pyp_nice_time() )
f = pyp_metrics.mating_coi(example.pedigree[4].animalID,
    example.pedigree[13].animalID,example,1)
print(f)
```

which produces the output:

```
Calling mating_coi() at Wed Mar  5 11:31:30 2008
0.25
```

We don't need `PyPedal` to tell us the coefficient of inbreeding of such a simple mating, but the calculations can be complex for more complicated cases. While you can do all of the necessary computations “by hand”, `pyp_metrics.mating_coi()` takes care of that for you by adding a new dummy animal to the pedigree with the proposed parents, calculating the coefficient of inbreeding of that mating, deleting the dummy animal from the pedigree, and returning the coefficient of inbreeding.

`PyPedal` takes things one step further, allowing you to work with groups of proposed matings at one time using `pyp_metrics.mating_coi_group()`. Internally it works much like `pyp_metrics.mating_coi()`, although instead of passing a pair of parents you pass a list of proposed matings. The matings are of the form `<parent1>_<parent2>`. Here's an example in which we're going to consider the following three matings: 1 with 5, 1 with 14, and 5 with 14. Note how the matings are formed and appended to the `matings` list in a single step.

```
matings = []
matings.append('%s_%s'%(example.pedigree[0].animalID, example.pedigree[4].animalID))
matings.append('%s_%s'%(example.pedigree[0].animalID, example.pedigree[13].animalID))
matings.append('%s_%s'%(example.pedigree[4].animalID, example.pedigree[13].animalID))
fgrp = pyp_metrics.mating_coi_group(matings,example)
```

```
print('fgrp: ', fgrp['matings'])
```

That code produces a list of the proposed matings with associated coefficients of inbreeding. In addition to the `matings` dictionary, the `fgrp` dictionary also contains a dictionary named `metadata` which contains summary statistics about the proposed matings.

```
fgrp: {'1_5': 0.25, '5_14': 0.25, '1_14': 0.125}
```

8.4 Relatives

PyPedal provides many tools for extracting information about relatives from a pedigree. Examples include: obtaining lists of the ancestors of an animal, getting lists of ancestors shared in common by a pair of animals, listing the descendants of an individual, calculating the additive genetic relationship between a given pair of animals, and the creation of “subpedigrees” containing only specified animals.

It’s easy to obtain a list of an animal’s relatives using `pyp_metrics.related_animals()`:

```
list_a = pyp_metrics.related_animals(example.pedigree[6].animalID,example)
list_b = pyp_metrics.related_animals(example.pedigree[13].animalID,example)
```

produces a list of each animal’s relatives:

```
[5, 1, 2]
[14, 5, 1, 2, 6, 3, 4]
```

`pyp_metrics.common_ancestors()` is used to get a list of the common ancestors of two animals, say animals 5 and 14 (remember that this pedigree is renumbered, and Python lists are indexed from 0, so we need to offset the animal IDs by 1 to get the correct animal IDs).

```
list_r = pyp_metrics.common_ancestors(example.pedigree[4].animalID,example.pedigree[13].an
malID,example)
print(list_r)
\end{verbatim}
Results are returned as lists:
\begin{verbatim}
[1, 2, 5]
```

If you’ve already obtained ancestor lists for a given pair of animals in which you’re interested you can also obtain a list of common ancestors using Python sets, which avoids performing more calculations than needed:

```

>>> set_a = set(list_a)
>>> set_b = set(list_b)
>>> set_c = set_a.intersection(set_b)
>>> set_c
set([1, 2, 5])
>>> list_c = list(set_c)
>>> list_c
[1, 2, 5]

```

A list of descendants can be obtained by calling `pyp_metrics.descendants()`, which returns a dictionary:

```

>>> pyp_metrics.descendants(5,example,{})
{7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14}

```

However, it's important to note that you won't get the answer you expect unless you have either set the option `set_offspring = 1` or called `pyp_utils.set_offspring()` after loading the pedigree. There's also a convenience function, `pyp_metrics.founder_descendants()`, for handling the special case of obtaining descendants of all the founders in the pedigree:

```

>>> pyp_metrics.founder_descendants(example)
{1: {5: 5, 7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14},
 2: {5: 5, 7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14},
 3: {6: 6, 7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14},
 4: {6: 6, 7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14}}

```

8.5 Decomposition and Direct Inverses of Numerator Relationship Matrices

PyPedal provides routines in the `pyp_nrm` module for decomposing and forming direct inverses of numerator relationship matrices (NRM). The NRM, A , may be written as $A = TDT'$, where T is a lower triangular matrix and D is a diagonal matrix (Henderson 1976; Thompson 1977). T traces the flow of genes from one generation to another, while D provides variances and covariances of Mendelian sampling. Mrode (1996) presents a more detailed discussion of this decomposition. The examples in this section all use the pedigree presented in Table 2.1 of that work, which is contained in the file `examples/new_decompose.ped`:

```

# Pedigree from Table 2.1 of
# Mrode (1996)
1 0 0

```

```

2 0 0
3 1 2
4 1 0
5 4 3
6 5 2

```

The results presented below may be obtained by running the `new_decompose.py` example program. The `pyp_nrm.a_decompose()` function takes as its argument a pedigree object and returns the matrices T and D . The code

```

print('Calling a_decompose()')
print('=====')
D, T = pyp_nrm.a_decompose(example)
print('D: ', D)
print()
print('T: ', T)

```

produces identical answers to those presented by Mrode (1996) (note that outputs may have been reformatted slightly to improve readability).

```

Calling a_decompose()
=====
D:  [[ 1.      0.      0.      0.      0.      0.      ]
 [ 0.      1.      0.      0.      0.      0.      ]
 [ 0.      0.      0.5      0.      0.      0.      ]
 [ 0.      0.      0.      0.75     0.      0.      ]
 [ 0.      0.      0.      0.      0.5      0.      ]
 [ 0.      0.      0.      0.      0.      0.46875]]

T:  [[ 1.      0.      0.      0.      0.      0.      ]
 [ 0.      1.      0.      0.      0.      0.      ]
 [ 0.5     0.5     1.      0.      0.      0.      ]
 [ 0.5     0.      0.      1.      0.      0.      ]
 [ 0.5     0.25    0.5     0.5     1.      0.      ]
 [ 0.25    0.625   0.25    0.25    0.5     1.      ]]

```

It's not generally feasible to directly compute the inverse, A^{-1} , of large NRM. However, it is possible to formulate simple rules for computing the inverse directly from. Henderson (1976) presented a simple procedure for calculating A^{-1} without accounting for inbreeding taking advantage of the fact that $A^{-1} = (T^{-1})'D^{-1}T^{-1}$, which is implemented as `pyp_nrm.form_d_nof()`:

```

print('Calling a_inverse_dnf()')
print('=====')
Ainv = pyp_nrm.a_inverse_dnf(example)
print('Ainv: ', Ainv)

```

produces A^{-1} without accounting for inbreeding in the pedigree:

```

Calling a_inverse_dnf()
=====
Ainv: [[ 1.83333333  0.5         -1.         -0.66666667  0.         0.
 [ 0.5         2.         -1.         0.         0.5         -1.
 [-1.         -1.         2.5         0.5         -1.         0.
 [-0.66666667  0.         0.5         1.83333333 -1.         0.
 [ 0.         0.5         -1.         -1.         2.5         -1.
 [ 0.         -1.         0.         0.         -1.         2.

```

Quaas (1976) built on Henderson's earlier work by providing an algorithm for the computation of A^{-1} accounting for inbreeding, which is implemented as the function `pyp_nrm.a_inverse_df()`.

```

print('Calling a_inverse_df()')
print('=====')
Ainv = pyp_nrm.a_inverse_df(example)
print('Ainv: ', Ainv)

```

You'll note the numbers in this matrix are slightly different from those in the previous example. Those differences are due to the inbreeding in the pedigree, which was not accounted for by `a_inverse_dnf()`.

```

Calling a_inverse_df()
=====
Ainv: [[ 1.83333333  0.5         -1.         -0.66666667  0.         0.
 [ 0.5         2.03333333 -1.         0.         0.53333333 -1.06666667]
 [-1.         -1.         2.5         0.5         -1.         0.
 [-0.66666667  0.         0.5         1.83333333 -1.         0.
 [ 0.         0.53333333 -1.         -1.         2.53333333 -1.06666667]
 [ 0.         -1.06666667  0.         0.         -1.06666667  2.13333333]]

```

The correctness of the results from `a_inverse_df()` can be verified by using some linear algebra to compute A^{-1} directly:

```

print('Calculating Ainv from D and T')
print('=====')
l = example.metadata.num_records
Tinv = numpy.linalg.inv(T)
print('Tinv: ', Tinv)
Tpinv = numpy.linalg.inv(T.T)
print('Tpinv: ', Tpinv)
Dinv = numpy.linalg.inv(D)
print('Dinv: ', Dinv)
Ainvhalf = numpy.dot(Tpinv,Dinv)
Ainv = numpy.dot(Ainvhalf,Tinv)
print('Ainv: ', Ainv)

```

The output includes several intermediate matrices that have been verified against Mrode's book (Mrode 1996).

```

Calculating Ainv from D and T
=====
Tinv: [[ 1.  0.  0.  0.  0.  0. ]
 [ 0.  1.  0.  0.  0.  0. ]
 [-0.5 -0.5  1.  0.  0.  0. ]
 [-0.5  0.  0.  1.  0.  0. ]
 [ 0.  0. -0.5 -0.5  1.  0. ]
 [ 0. -0.5  0.  0. -0.5  1. ]]

Tpinv: [[ 1.  0. -0.5 -0.5  0.  0. ]
 [ 0.  1. -0.5  0.  0. -0.5]
 [ 0.  0.  1.  0. -0.5  0. ]
 [ 0.  0.  0.  1. -0.5  0. ]
 [ 0.  0.  0.  0.  1. -0.5]
 [ 0.  0.  0.  0.  0.  1. ]]

Dinv: [[ 1.  0.  0.  0.  0.  0. ]
 [ 0.  1.  0.  0.  0.  0. ]
 [ 0.  0.  2.  0.  0.  0. ]
 [ 0.  0.  0.  1.33333333  0.  0. ]
 [ 0.  0.  0.  0.  2.  0. ]
 [ 0.  0.  0.  0.  0.  2.13333333]]

Ainv: [[ 1.83333333  0.5 -1. -0.66666667  0.  0. ]
 [ 0.5  2.03333333 -1.  0.  0.53333333 -1.06666667]]

```

```

[-1.      -1.      2.5      0.5      -1.      0.      ]
[-0.66666667  0.      0.5      1.83333333 -1.      0.      ]
[ 0.      0.53333333 -1.      -1.      2.53333333 -1.06666667]
[ 0.      -1.06666667  0.      0.      -1.06666667  2.13333333]]

```

The function `pyp_nrm.form_d_nof()` is provided as a convenience function. The form of D is the same when calculating A^{-1} regardless of the form T takes (i.e., accounts for inbreeding or not). The code

```

print('Calling form_d_nof()')
print('=====')
D = pyp_nrm.form_d_nof(example)
print('D: ', D)

```

produces the result

```

Calling form_d_nof()
=====
D:  [[ 1.      0.      0.      0.      0.      0.      ]
      [ 0.      1.      0.      0.      0.      0.      ]
      [ 0.      0.      0.5  0.      0.      0.      ]
      [ 0.      0.      0.      0.75  0.      0.      ]
      [ 0.      0.      0.      0.      0.5  0.      ]
      [ 0.      0.      0.      0.      0.      0.5  ]]
```

which may be verified against the values of D presented above.

8.6 Pedigrees as Sets

PyPedal has new tools for working with pedigrees by treating them as mathematical sets of animals. This makes it easy to do things like merge pedigrees, or remove a subset of animals from a pedigree. Formally, we can think of a pedigree, P , as a set of n animals denoted $a_1, a_2, \dots, a_n$. Two animals, a_i and a_j , are related iff $a_i, a_j \in P$. Two pedigrees, P and Q , may then be merged by taking the union of the two sets, $R = P \cup Q$, using the `NewPedigree::union()` method or `pyp_utils.list_union()` functions. Similarly, a subset of animals, Q , can be “subtracted” from a larger pedigree, P , as $R = P - Q = P - (P \cap Q)$.¹

A pair of pedigrees also can be merged using the `NewPedigree::__add__()` method, which provides an “addition” operator for `NewPedigree` instances, as shown in the following code:

¹Stated here without proof is the assertion that if you begin a null pedigree, that is, $P = \emptyset$, all possible pedigrees can be constructed using only unions ($\cup$) and intersections ($\cap$) of P with individual animals.


```

options = {}
options['pedname'] = 'Fake Pedigree 1'
options['renumber'] = 1
options['pedfile'] = 'merge1.ped'
options['pedformat'] = 'asd'
merge1 = pyp_newclasses.loadPedigree(options)

options2 = {}
options2['pedname'] = 'Fake Pedigree 2'
options2['renumber'] = 1
options2['pedfile'] = 'merge2.ped'
options2['pedformat'] = 'asd'
merge2 = pyp_newclasses.loadPedigree(options2)

merge3 = merge1 + merge2

```

The `NewPedigree::__sub__()` method also provide a “subtraction” operator for subtracting the animals in one pedigree from another, as described above.

Note that the `pyp_utils.list_union()` method takes advantage of the fact that $A \cap B \cap \dots \cap Y \cap Z$ can be factorized as $(A \cap (B \cap (\dots \cap (Y \cap Z))))$.

9 NAME: objects.qmd

In every chaotic behaviour, there lies a pattern. — Jean Jacques Rousseau

In this chapter, a detailed explanation of each PyPedal class is presented, including attributes and methods.

9.1 Animal Objects

Three types of animal object are provided in PyPedal (Section 9.1.1). Users will typically work with instances of `NewAnimal` objects, while `LightAnimal` (`?@sec-objects-animal-objects-light-animal`) and `SimAnimal` (`?@sec-objects-animal-objects-sim-animal`) objects are of interest primarily to developers. Detailed descriptions of each class and its methods may be found in the API Reference for the `pyp_newclasses` module (`?@sec-functions-pyp-newclasses`).

9.1.1 The NewAnimal Class

Table 9.1: Attributes of `NewAnimal` objects..

| At-tribute | Defaults for integer IDs (<code>asd</code>) | Defaults for string IDs (<code>ASD</code>) | Description |
|------------|---|--|--|
| age | -999 | -999 | The animal's age based on the global <code>BASE_DEMOGRAPHIC_YEAR</code> defined in <code>pyp_demog</code> . If the <code>by</code> is unknown, the inferred generation is used. If the inferred generation is unknown, the age is set to -999. |

`NewAnimal` objects have the seven methods listed in `?@tbl-objects-animal-objects-new-animal-methods`. The methods focus on returning information about an instance of an object; calculations are left to functions in, e.g., the `pyp_metrics` `pyp_nrm` modules.

10 NAME: methodology.qmd

To iterate is human, to recurse divine. — L. Peter Deutsch

In this chapter, a high-level overview of PyPedal is provided, giving the reader the definitions of the key components of the system. This section defines the concepts used by the remaining sections.

10.1 Reordering and Renumbering

Many computations on pedigrees require that the pedigree be renumbered such that animal IDs are consecutive from 1 to n , where n is the total number of animals in the pedigree. By default, all pedigrees are renumbered at load time. If your pedigree is already renumbered and you don't want it renumbered again you must set the **renumber** option to 0. The renumbering process requires that the pedigree be reordered such that parents always precede their offspring in the list of animal IDs. The actual ID assigned to an animal is of no particular importance, and it is even possible for parents to have larger IDs than their offspring. PyPedal can reorder any pedigree unless there is an error in it that would prevent unambiguously placing parents before offspring. For example, a pedigree containing a keypunch error such that an animal is one of its own grandparents cannot be reordered because there is no way to unambiguously order the animals. The `pyp_utils` module provides two routines for pedigree reordering, `reorder()` and `fast_reorder()`. By default, `reorder()` is used to reorder pedigrees in place. It does this by maintaining a list of animal IDs that have been processed; whenever a parent that isn't in the list of encountered animals is found, the offspring of that parent are moved to the end of the pedigree. This ensures the pedigree is properly sorted such that all parents precede their offspring. Founders are also grouped together at the beginning of the pedigree. This procedure will always correctly reorder a pedigree but it can be quite inefficient as it is similar to an insertion sort, which has a worst-case runtime proportional to n^2 (Cormen et al. 2003).

`fast_reorder()` provides a much faster means of reordering a pedigree, but can incorrectly reorder a pedigree in some cases. When an instance of a `NewAnimal` object is created the `pad_id()` method is called. `pad_id()` uses the animal ID and birth year to form an ID used by `pyp_utils/fast_reorder()` for quick sorting; if your pedigree file is numbered such that offspring always have larger IDs than their parents and your birth years (if provided) are correct (that is, parents always born BEFORE offspring) then `pyp_utils.fast_reorder()`

works as expected. If you don't provide birth years in your pedigree file but your parent IDs are always smaller than your animal IDs, the reordering will be correct. If you do not provide birth years, all animals in the pedigree will be assigned a default value of 1900'. In that case, if parents have IDs larger than that of one or more of their offspring, the pedigree will be incorrectly reordered by `fast_reorder()`. If your pedigree file contains birth years, or you know that parents always have smaller IDs than their offspring, then `fast_reorder()` will correctly reorder your pedigree in linear time. Founders are not guaranteed to be grouped at the beginning of the pedigree when `fast_reorder()` is used; if you're going to calculate coefficients of partial inbreeding (`@sec-methodology-partial-inbreeding`) then you should instead use `reorder()` to reorder your pedigree.

The performance difference between the two reordering routines is not very noticeable on pedigrees of a few hundred to a few thousand animals, but is quite dramatic for very large pedigrees. If your pedigree file is already reordered then there's essentially no performance difference between the two. When creating a pedigree file from data stored in a relational database, let the database perform the sort for you by using an `ORDER BY` statement.

10.2 Animal Identification and Cross-References

There are a number of identification attributes associated with animal objects in PyPedal pedigrees. A description of those fields, as well as their default values, is provided in Table 9.1. This section describes the data structures provided for mapping between various animal IDs. Table 10.1 lists the four structures provided for ID mapping and lists the default values for pedigrees with integer (`asd` formats) and string (`ASD` formats) IDs. Renumbered PyPedal pedigree objects store animal objects in a list that is typically indexed by renumbered ID. When animal IDs are strings (see Section [?@sec-pedigree-format-codes](#)) they're hashed to an integer and the original ID is stored in a `name` field. In renumbered pedigrees the original ID is stored and replaced by a renumbered ID.

Once your pedigree is renumbered it's quite easy to see how these maps can be used to convert between various IDs. The maps mean you don't have to worry about renumbered IDs and can continue to think about your animals in terms of their original IDs, whether they be ID numbers or names. Consider the Newfoundland pedigree presented in Figure [?@fig-newfoundland-colored-pedigree](#) – it's much more convenient to think about the dog named Kaptn Kvols von Widdersdorf, rather than the dog whose name was hashed to the ID 5523557808241831142 and renumbered to 48. For example, suppose you wanted to determine his coefficient of inbreeding. It's simple to do using the maps:

`example.namemap['Kaptn Kvols von Widdersdorf']` returns the original ID assigned to the name, while `example.idmap[...]` converts from the original ID to the renumbered ID. This sort of ID/name mapping is used in a number of places in PyPedal, such as in the three-generation pedigree routine in the `pyp_reports` module.

Note that if an animal has its original ID as its name, which is the default when integral IDs and no animal names are provided, the name is changed to the renumbered ID when the pedigree is renumbered.

Table 10.1: Animal identification and cross-references.

| | Key | Value | Key | Value | Key | Value |
|--------------|-----------|-----------|------|-------|---------------|---------------|
| idmap | animal ID | animal ID | name | name | original ID | renumbered ID |
| backmap | animal ID | animal ID | name | name | renumbered ID | original ID |
| namemap | name | animal ID | name | name | name | original ID |
| name-backmap | animal ID | name | name | name | original ID | name |

10.3 Measures of Genetic Variation

Coefficients of inbreeding and relationship (Wright 1922) have been commonly used to describe the genetic diversity in livestock populations (Young and Seykora 1996). Inbreeding coefficients represent an individual's expected genetic homozygosity due to the relatedness of its parents. Coefficients of relationship describe the expected proportion of genes two individuals share due to their relatedness. These are relative measures that depend on such factors as the completeness and depth of pedigrees. Over time, these coefficients change in response to breeding and culling decisions, and they may be used as indicators of the genetic variability of a population. Rapid methods for calculating coefficients of inbreeding and relationship for large populations have been implemented (Wiggans, Van Raden, and Zuurbier 1995).

Populations under study rarely conform to the theory established for the use of coefficients of inbreeding (Wright 1931). Lacy (1989) and Boichard et al. (1997) proposed measures of genetic variation based on ideas from conservation genetics. Lacy (1989) proposed the idea of the number of founder equivalents in assessing populations. A founder is an ancestor whose parents are unknown. If all founders contribute to the population equally, then the founder equivalent is equal to the number of founders. When founders contribute unequally to the population, the number of founder equivalents decreases. Boichard et al. (1997) developed the idea of founder ancestor equivalents, which is the minimum number of ancestors necessary to explain the genetic diversity of the current population. Founder ancestor equivalents account for bottlenecks, unlike founder equivalents, and are more accurate in populations undergoing intense selection. Caballero and Toro (2000) discussed the relationships among these and other measures of diversity in small populations, and demonstrate their use (Toro et al. 2000).

Roughsedge et al. (1999) used average coefficients of inbreeding, average coefficients of relationship, founder equivalent numbers, and founder ancestor numbers to document the decrease in genetic diversity in the British dairy cattle population over the last 25 years. Changes in founder equivalent number and founder ancestor number reflected the use of a small

number of influential individuals to improve the genetic merit of that population. Accompanying changes in average inbreeding and relationship did not accurately reflect that loss of diversity. Such results highlight the need for additional tools when assessing complex populations.

10.4 Computational Details

Coefficients of relationship (r_{ij}) and inbreeding (f_i) are calculated using the method of Wiggans et al. (1995). An empty dictionary is created to store animal IDs and coefficients of inbreeding. For each animal in the pedigree, working from youngest to oldest, the dictionary is queried for the animal ID. If the animal does not have an entry in the dictionary, a subpedigree containing only relatives of that animal is extracted and the coefficients of inbreeding are calculated and stored in the dictionary. A second dictionary keeps track of sire-dam combinations seen in the pedigree. If a full-sib of an animal whose pedigree has already been processed is encountered the full-sib receives a COI identical to that of the animal already processed. This approach allows for computation of COI for arbitrarily large populations because it doesn't require allocation of a single NRM of order n^2 , where n is the size of the pedigreed population. In most cases, the NRM for a subpedigree is on the order of 200, although this can vary with species and population data structure.

Average and maximum coefficients of inbreeding are computed for the entire population and for all individuals with non-zero inbreeding. The average relationship among all individuals is also computed. Theoretical and realized effective population sizes, $N_{e(t)}$, and $N_{e(r)}$, were estimated as (Falconer and MacKay 1996):

$$N_{e(t)} = \frac{4N_m N_f}{N_m + N_f}$$

and

$$N_{e(t)} = \frac{1}{2\Delta f}$$

where N_m and N_f are the number of sires and dams in the population, respectively, and Δf is the change in population average inbreeding between generations t and $t+1$. Interpretation of $N_{e(t)}$ can be problematic when Δf is calculated from incomplete or error-prone pedigrees.

10.5 Ancestral Inbreeding

Ballou (1997) described ancestral inbreeding, the probability that an individual inherited an allele that had undergone inbreeding in the past at least once, in a study of purging recessives and inbreeding in conservation genetics. This is a different idea than the usual coefficient of inbreeding as an individual that is not inbred may carry alleles that have been exposed to substantial inbreeding; recall that an individual may have inbred parents, but if the parents are not related to one another then the resulting offspring will not be themselves inbred. It has been proposed that animals deriving from highly inbred lines may be less susceptible to inbreeding depression because deleterious recessive alleles have been purged from the population, but literature reports on extant populations are inconsistent. Ancestral inbreeding is calculated using a recursion equation as:

$$f_a = [f_{a(s)} + (1 - f_{a(s)})f_s + f_{a(d)} + (1 - f_{a(d)})f_d]/2$$

where f_a is the ancestral inbreeding coefficient for an individual, f is the usual coefficient of inbreeding, and subscripts s and d represent sires and dams, respectively. Calculations are from oldest to youngest in the population and require as inputs only coefficients of inbreeding.

Suwanlee et al. (2007) extended the concept by presenting a gene-dropping approach (MacCluer et al. 1986) for calculating ancestral inbreeding, as well as modifying Ballou's equation to account for non-independence between individual inbreeding coefficients and ancestral inbreeding coefficients. Gene-dropping is also used in PyPedal for calculating founder genome equivalents (see: Section [?@sec-methodology-computation-founder-genome-equivalents](#)), and the code used by the `pyp_metrics.dropped_ancestral_inbreeding()` and `pyp_metrics.effective_founder_genomes()` routines are similar, although the former drops an arbitrary number of unlinked biallelic loci through the pedigree while the latter drops only a single locus.

10.6 Partial Inbreeding

Partial inbreeding coefficients, F_{ij} , measure the probability that the alleles at an arbitrary locus in individual i are identical-by-descent and that the alleles were derived from an allele in founder j [LacyAW1996]; Gulisija et al. (2006) provide an excellent description of the tabular method for calculating F_{ij} . Computational requirements may be high for large pedigrees with a large number of founders because partial kinship matrices are calculated for each founder in the pedigree. The usual coefficients of inbreeding may be obtained by summing the coefficients of partial inbreeding over all founders common to the parents of animal i , that is, $f_i = \sum_j F_{ij}$.

For example, consider the pedigree presented in Figure 2 of Gulisija and Crow (2007). The individual of interest, I , has an inbreeding coefficient of 0.375 and coefficients of partial

inbreeding to founders J, K, and M of 0.21875, 0.09375, and 0.0625, respectively. As asserted, $0.21875 + 0.09375 + 0.0625 = 0.375$.

10.7 Generation Coefficients

{sec-methodology-computation-generation-coefficients}

Generation coefficients are assigned using the method of [Pattie1965]. Founders, defined as individuals with unknown parents, are assigned generation codes of 0. All other animals are assigned generation codes as:

$$GC_o = \frac{(GC_s + GC_d)}{2} + 1$$

where GC_o , GC_s , GC_d represent offspring, sire, and dam codes, respectively.

10.8 Effective Founder Number

{sec-methodology-computation-effective-founder-number}

The effective founder number (f_e) was calculated as:

$$f_e = \frac{1}{\sum p_i^2}$$

where p_i is the proportion of genes contributed by ancestor i to the current population (Lacy 1989). If all founders had contributed equally to the population, then f_e would be the same as the actual number of founders. When founders contribute to the population unequally, f_e is smaller than the actual number of founders. The greater the inequity in founder contributions, the smaller the effective founder number.

A subpedigree approach, similar to that used for calculation of inbreeding (see {sec-methodology-computation-inbreeding} for details), is also used for calculating f_e .

10.9 Founder Genome Equivalents

{sec-methodology-computation-founder-genome-equivalents}

Lacy (1989) also defined the number of founder genome equivalents (f_g) as a measure of genetic diversity. A founder genome equivalent is the number of founders that would produce a population with the same diversity of founder alleles as the pedigree population assuming all

founders contributed equally to each generation of descendants. Founder genome equivalents are calculated as:

$$f_g = \frac{1}{\sum \frac{p_i}{r_i}}$$

where p_i is the proportion of genes contributed by ancestor i to the current population and r_i is the proportion of founder i 's genes that are retained in the current population. Like f_e , f_g accounts for unequal founder contributions. Unlike f_e , f_g also accounts for the fraction of founder genomes lost from the pedigree through drift during bottlenecks. Although f_g is the more accurate description of the amount of founder variation present in a population, it can only be calculated directly for simple pedigrees. For large or complex pedigrees, the number of founder genome equivalents must be approximated based on computer simulation of a large number of segregations through the pedigree. This is done by assigning each founder a unique pair of alleles and randomly transmitting those alleles through the pedigree [ref1719]. The number of founder genome equivalents is similar to the effective founder number, but the former has been devalued based on the proportion of its genome that has probably been lost to drift [ref640].

10.10 Effective Ancestor Number

In populations that have undergone a bottleneck the effective number of founders computed using Lacy's (1989) approach is overestimated. Large contributions made by recent ancestors are more important to the population with respect to the loss of genetic diversity than equal contributions made long ago. Boichard et al. (1997) proposed a second measure of diversity to deal with such situations, the effective number of ancestors (f_a), which considers the genetic contribution of all ancestors in the population, not just founders. The effective number of ancestors treats all ancestors in the population the same way, and is computed as:

$$f_a = \frac{1}{\sum q_i^2}$$

where q_i is the genetic contribution of the i -th ancestor not explained by the previous $i - 1$ ancestors. The ancestors with the greatest contributions are selected iteratively. The number of ancestors with a positive genetic contribution is less than or equal to the actual number of founders.

10.11 Pedigree Completeness

{sec-methodology-pedigree-completeness}

Pedigree completeness [Cassell2003a], the proportion of known pedigree information for an arbitrary number of generations, is computed as:

$$c_p = \frac{a_k}{\sum_{i=1}^g 2^i}$$

where c_p is pedigree completeness and a_k is the number of known ancestors in g generations. The default (which may be changed) is to compute four-generation pedigree completeness. Low c_p indicates that there is little pedigree information available for an individual, which may result in biased estimates of inbreeding and other measures of diversity. Pedigree completeness and ancestor loss coefficients, which are sometimes seen in dog breeding materials, are equivalent measures if the same number of generations were used in the calculations.

Listing 10.1

```
from PyPedal import pyp_newclasses
from PyPedal import pyp_nrm

example = pyp_newclasses.load_pedigree(optionsfile='../test_files/newfoundland.ini')
newf_f = pyp_nrm.inbreeding(example)
print(newf_f['fx'][example.idmap[example.namemap['Kaptn Kvols von Widdersdorf']]])
```

```
[INFO]: Logfile untitled_pedigree.log instantiated.
[INFO]: Preprocessing ../test_files/newfoundland.ped
[INFO]: Opening pedigree file ../test_files/newfoundland.ped
[NOTE]: Added pedigree entry for sire Highland Bear of Pouch Cove
[NOTE]: Added pedigree entry for sire Schooners Yosef of Newton Ark
[NOTE]: Added pedigree entry for sire Amitys Bearfoot of Pouch Cove
[NOTE]: Added pedigree entry for sire Spokinewfs Cariboo Cowboy
[NOTE]: Added pedigree entry for sire Nordbjorns Gringo
[NOTE]: Added pedigree entry for sire Arvals Nordstrands Talisman
[NOTE]: Added pedigree entry for sire Hector van het Heidemeer
[NOTE]: Added pedigree entry for sire Fjordblinks Hakon
[NOTE]: Added pedigree entry for sire Aaron van het Hoogveen
[NOTE]: Added pedigree entry for sire Boedha van het Hoogveen
[NOTE]: Added pedigree entry for sire Enrik von Luxemburg
[NOTE]: Added pedigree entry for sire La Bellas Rigoletto
[NOTE]: Added pedigree entry for sire As-Dur vom Weserwall
[NOTE]: Added pedigree entry for sire Graaf Gabor van Overessinge
[NOTE]: Added pedigree entry for sire Benno von Ammertal
[NOTE]: Added pedigree entry for dam Ad Lib of Pouch Cove
[NOTE]: Added pedigree entry for dam Souvenir of Pouch Cove
[NOTE]: Added pedigree entry for dam Lifebuoys Penny of Bubbelinea
[NOTE]: Added pedigree entry for dam Jehaj Vilhelmina
[NOTE]: Added pedigree entry for dam Black Domes Miss Caminickers
[NOTE]: Added pedigree entry for dam Ellis-Bijou of the Thatch Roof
[NOTE]: Added pedigree entry for dam Fenja vom Riesrand
[NOTE]: Added pedigree entry for dam Gracia van de Papenhof
[NOTE]: Added pedigree entry for dam Dukeroys Daphne
[NOTE]: Added pedigree entry for dam Zarah von Luxemburg
[NOTE]: Added pedigree entry for dam Frederikke
[NOTE]: Added pedigree entry for dam Bonny vom Weserwall
[NOTE]: Added pedigree entry for dam Fleur of the Coastguard
[NOTE]: Added pedigree entry for dam Ina vom Forellenhof
[INFO]: Assigning sexes at Wed Dec 17 11:20:58 2025
[NOTE]: pyp_utils/set_sexes() assigning a sex to all animals in pedigree Newfoundland Pedigree
[INFO]: Creating pedigree metadata object
[INFO]: Instantiating a new PedigreeMetadata() object...
[INFO]: Naming the Pedigree()...
[INFO]: Assigning a filename...
[INFO]: Attaching a pedigree...
[INFO]: Setting the pedcode...
[INFO]: Counting the number of animals in the pedigree...
[INFO]: Counting and finding unique sires...
[INFO]: Counting and finding unique dams...
```

References

- Ballou, J. D. 1997. "Ancestral Inbreeding Only Minimally Affects Inbreeding Depression in Mammalian Populations." *Journal of Heredity* 88: 169–78.
- Boichard, D., L. Maignel, and E. Verrier. 1997. "The Value of Using Probabilities of Gene Origin to Measure Genetic Variability in a Population." *Genetics Selection Evolution* 29: 5–23.
- Caballero, A., and M. A. Toro. 2000. "Interrelations Between Effective Population Size and Other Pedigree Tools for the Management of Conserved Populations." *Genetical Research (Cambridge)* 75: 331–43.
- Cole, J. B. 2007. "PyPedal: A Computer Program for Pedigree Analysis." *Computers and Electronics in Agriculture* 57: 107–13.
- Cole, J. B., D. E. Franke, and E. A. Leighton. 2004. "Population Structure of a Colony of Dog Guides." *Journal of Animal Science* 82: 2906–12.
- Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein. 2003. *Introduction to Algorithms, Second Edition*. Englewood Cliffs, NJ: Prentice-Hall.
- Falconer, D. S., and F. C. MacKay. 1996. *Introduction to Quantitative Genetics*. 4th ed. Longman.
- Gulisija, D., and J. F. Crow. 2007. "Inferring Purging from Pedigree Data." *Evolution* 61: 1043–51.
- Gulisija, D., D. Gianola, K. A. Weigel, and M. A. Toro. 2006. "Between-Founder Heterogeneity in Inbreeding Depression for Production in Jersey Cows." *Livestock Science* 104: 244–53.
- Henderson, C. R. 1976. "A Simple Method for Computing the Inverse of a Numerator Relationship Matrix Used in Prediction of Breeding Values." *Biometrics* 32: 69–83.
- Lacy, R. C. 1989. "Analysis of Founder Representation in Pedigrees: Founder Equivalents and Founder Genome Equivalents." *Zoo Biology*. 8: 111–23.
- Lacy, R. C., G. Alaks, and A. Walsh. 1996. "Hierarchical Analysis of Inbreeding Depression in *Peromyscus Polionotus*." *Evolution* 50: 2187–2200.
- MacCluer, J. W., J. L. VandeBerg, B. Read, and O. A. Ryder. 1986. "Pedigree Analysis by Computer Simulation." *Zoo Biology*. 5: 147–60.
- Meuwissen, H. E., and Z. Luo. 1992. "Computing Inbreeding Coefficients in Large Populations." *Genetics Selection Evolution* 24: 305–13.
- Mrode, R. A. 1996. *Linear Models for the Prediction of Animal Breeding Values*. Wallingford, UK: CAB International.
- Quaas, R. L. 1976. "Computing the Diagonal Elements and Inverse of a Large Numerator Relationship Matrix." *Biometrics* 32: 949–53.

- Roughsedge, T., S. Brotherstone, and P. M. Visscher. 1999. "Quantifying Genetic Contributions to a Dairy Cattle Population Using Pedigree Analysis." *Livestock Production Science* 60: 359–69.
- Suwanlee, S., R. Baumung, J. Sölkner, and I. Curik. 2007. "Evaluation of Ancestral Inbreeding Coefficients: Ballou's Formula Versus Gene Dropping." *Conservation Genetics* 8: 489–95.
- Thompson, R. 1977. "The Estimation of Heritability with Unbalanced Data i. Observations Available on Parents and Offspring." *Biometrics* 33: 495.
- Toro, M. A., J. Rodriganez, L. Silio., and C. Rodriguez. 2000. "Genealogical Analysis of a Closed Herd of Black Hairless Iberian Pigs." *Conservation Biology* 14.
- Wiggans, G. R., P. M. Van Raden, and J. Zuurbier. 1995. "Calculation and Use of Inbreeding Coefficients for Genetic Evaluation of United States Dairy Cattle." *Journal of Dairy Science* 78: 1584–90.
- Wright, S. 1922. "Coefficients of Inbreeding and Relationship." *Amer. Nat.* 56: 330–38.
- . 1931. "Evolution in Mendelian Populations." *Genetics* 16: 97–159.
- Young, C. W., and A. J. Seykora. 1996. "Estimates of Inbreeding and Relationship Among Registered Holstein Females in the United States." *Journal of Dairy Science* 79: 502–5.